

HerAI Fellowship

FRONT-END DEVELOPMENT FOR AI PRODUCTS

Modul Belajar Praktis dan Mudah Dipahami

Membangun antarmuka AI yang responsif, transparan, inklusif, dan memberi kendali kepada pengguna.

Level	Beginner to Intermediate
Estimasi belajar	18-22 jam
Prasyarat	Logika pemrograman dan dasar penggunaan web
Versi	1.0 - 23 Juli 2026
Route	/participant/courses/front-end-development-for-ai

Disusun untuk Participant Module - Project HerAI

Tentang Modul

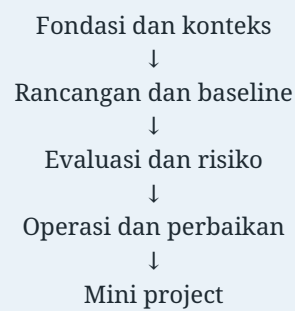
Modul ini menggabungkan fondasi front-end dengan pola interaksi khusus AI: loading yang panjang, streaming, ketidakpastian, koreksi, upload multimodal, citation, dan human control.

Pola belajar: pahami tujuan, buat baseline kecil, uji kegagalan, dokumentasikan bukti, lalu perbaiki secara bertahap. Jangan menambah kompleksitas sebelum masalah dan metriknya jelas.

Cara Menggunakan Modul

- Baca tujuan dan gambaran sederhana setiap bab.
- Kerjakan checkpoint tanpa melihat kembali materi.
- Adaptasi contoh pada data aman atau sintetis.
- Catat asumsi, error, keputusan, dan keterbatasan.
- Gunakan mini project sebagai artefak portofolio.

Peta Belajar



Daftar Isi

01 Bab 1 - Peran Front-end dan Karakteristik AI UX

02 Bab 2 - Fondasi HTML, CSS, dan JavaScript

03 Bab 3 - Component, State, dan Data Flow

04 Bab 4 - Integrasi API dan Async Interaction

05 Bab 5 - Chat, Streaming, dan Progressive Rendering

06 Bab 6 - Upload Gambar, Audio, dan Dokumen

07 Bab 7 - Accessibility dan Internationalization

08 Bab 8 - Keamanan dan Privasi di Browser

09 Bab 9 - Testing dan Performa Front-end

10 Bab 10 - Design System dan Product Analytics

11 Bab 11 - Mini Project: Dashboard Evaluasi Diri Berbasis Umpan Balik

12 Bab 12 - Kuis Akhir

13 Bab 13 - Diskusi dan Refleksi

14 Bab 14 - Glosarium dan Checklist

15 Bab 15 - Ringkasan dan Referensi

HerAI Fellowship - Participant Module Membangun antarmuka AI yang responsif, transparan, inklusif, dan memberi kendali kepada pengguna.

Tentang Modul

Modul ini menggabungkan fondasi front-end dengan pola interaksi khusus AI: loading yang panjang, streaming, ketidakpastian, koreksi, upload multimodal, citation, dan human control.

Modul disusun dengan bahasa bertahap. Setiap bab berisi tujuan, konsep inti, alur kerja, contoh kasus, kesalahan umum, checkpoint, dan latihan. Peserta tidak perlu menghafal seluruh istilah. Yang lebih penting adalah memahami hubungan antarkomponen dan mampu menjelaskan alasan di balik sebuah pilihan.

Capaian Pembelajaran

Setelah menyelesaikan modul, peserta mampu:

- menjelaskan fondasi dan istilah utama Front-end Development for AI Products;
- menerjemahkan kebutuhan menjadi rancangan yang dapat diuji;
- membuat prototipe kecil dengan dokumentasi dan kontrol dasar;
- mengevaluasi kualitas, risiko, keterbatasan, serta biaya;
- menyampaikan hasil dan rekomendasi kepada pemangku kepentingan.

Prasyarat

- Logika pemrograman dan dasar penggunaan web.
- Mampu membaca diagram sederhana dan mengikuti contoh kode.
- Bersedia mencatat asumsi, kegagalan, dan keputusan selama latihan.

Cara Belajar yang Disarankan

1. Baca gambaran dan tujuan setiap bab.
2. Buat catatan istilah dengan bahasa sendiri.
3. Kerjakan checkpoint sebelum melihat kembali materi.
4. Jalankan atau adaptasi contoh kode bila relevan.
5. Gunakan mini project sebagai portofolio, bukan sekadar tugas akhir.
6. Lakukan review keamanan, privasi, dan dampak sebelum demonstrasi.

Peta Materi

Tahap	Fokus	Hasil
Fondasi	Istilah, tujuan, konteks, dan arsitektur	Peta masalah yang jelas
Pembangunan	Data, proses, komponen, dan integrasi	Baseline yang dapat diuji
Evaluasi	Kualitas, risiko, subgroup, biaya	Bukti dan daftar keterbatasan
Operasi	Monitoring, ownership, respons, iterasi	Sistem yang dapat dipelihara

Bab 1 - Peran Front-end dan Karakteristik AI UX

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran front-end dan karakteristik ai ux dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Peran Front-end dan Karakteristik AI UX tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
interface	konsep penting dalam peran front-end dan karakteristik ai ux yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana interface diukur, diuji, atau dikendalikan?
feedback	konsep penting dalam peran front-end dan karakteristik ai ux yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana feedback diukur, diuji, atau dikendalikan?
uncertainty	tingkat ketidakpastian yang perlu ditampilkan dan dikelola	Pertanyaan yang perlu dijawab: bagaimana uncertainty diukur, diuji, atau dikendalikan?
control	konsep penting dalam peran front-end dan karakteristik ai ux yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana control diukur, diuji, atau dikendalikan?
trust	konsep penting dalam peran front-end dan karakteristik ai ux yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana trust diukur, diuji, atau dikendalikan?
failure	konsep penting dalam peran front-end dan karakteristik ai ux yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana failure diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **interface** -> **feedback** -> **uncertainty** -> **control**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **interface** dapat memengaruhi **feedback**, lalu mengubah cara **uncertainty** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan peran front-end dan karakteristik ai ux dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk interface serta feedback.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan peran front-end dan karakteristik ai ux secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **interface** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan peran front-end dan karakteristik ai ux.
- Menganggap interface sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan interface dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara feedback dan uncertainty dalam bab ini?
3. Sebutkan satu risiko ketika peran front-end dan karakteristik ai ux diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan interface, feedback, uncertainty, control. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 2 - Fondasi HTML, CSS, dan JavaScript

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran fondasi html, css, dan javascript dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Fondasi HTML, CSS, dan JavaScript tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
semantic HTML	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana semantic HTML diukur, diuji, atau dikendalikan?
layout	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana layout diukur, diuji, atau dikendalikan?
responsive	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana responsive diukur, diuji, atau dikendalikan?
DOM	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana DOM diukur, diuji, atau dikendalikan?
event	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana event diukur, diuji, atau dikendalikan?
module	konsep penting dalam fondasi html, css, dan javascript yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana module diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **semantic HTML** -> **layout** -> **responsive** -> **DOM**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **semantic HTML** dapat memengaruhi **layout**, lalu mengubah cara **responsive** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan fondasi html, css, dan javascript dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk semantic HTML serta layout.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan fondasi html, css, dan javascript secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **semantic HTML** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan fondasi html, css, dan javascript.
- Menganggap semantic HTML sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan semantic HTML dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara layout dan responsive dalam bab ini?
3. Sebutkan satu risiko ketika fondasi html, css, dan javascript diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan semantic HTML, layout, responsive, DOM. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 3 - Component, State, dan Data Flow

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran component, state, dan data flow dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Component, State, dan Data Flow tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
component	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana component diukur, diuji, atau dikendalikan?
props	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana props diukur, diuji, atau dikendalikan?
state	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana state diukur, diuji, atau dikendalikan?
derived state	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana derived state diukur, diuji, atau dikendalikan?
lifecycle	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana lifecycle diukur, diuji, atau dikendalikan?
architecture	konsep penting dalam component, state, dan data flow yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana architecture diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **component** -> **props** -> **state** -> **derived state**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **component** dapat memengaruhi **props**, lalu mengubah cara **state** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan component, state, dan data flow dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk component serta props.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan component, state, dan data flow secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **component** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan component, state, dan data flow.
- Menganggap component sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan component dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara props dan state dalam bab ini?
3. Sebutkan satu risiko ketika component, state, dan data flow diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan component, props, state, derived state. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 4 - Integrasi API dan Async Interaction

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran integrasi api dan async interaction dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Integrasi API dan Async Interaction tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
fetch	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana fetch diukur, diuji, atau dikendalikan?
promise	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana promise diukur, diuji, atau dikendalikan?
loading	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana loading diukur, diuji, atau dikendalikan?
timeout	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana timeout diukur, diuji, atau dikendalikan?
retry	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana retry diukur, diuji, atau dikendalikan?
cancellation	konsep penting dalam integrasi api dan async interaction yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana cancellation diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **fetch** -> **promise** -> **loading** -> **timeout**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **fetch** dapat memengaruhi **promise**, lalu mengubah cara **loading** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan integrasi api dan async interaction dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk fetch serta promise.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan integrasi api dan async interaction secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **fetch** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan integrasi api dan async interaction.
- Menganggap fetch sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan fetch dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara promise dan loading dalam bab ini?
3. Sebutkan satu risiko ketika integrasi api dan async interaction diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan fetch, promise, loading, timeout. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 5 - Chat, Streaming, dan Progressive Rendering

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran chat, streaming, dan progressive rendering dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Chat, Streaming, dan Progressive Rendering tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
stream	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana stream diukur, diuji, atau dikendalikan?
token	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana token diukur, diuji, atau dikendalikan?
typing state	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana typing state diukur, diuji, atau dikendalikan?
stop	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana stop diukur, diuji, atau dikendalikan?
regenerate	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana regenerate diukur, diuji, atau dikendalikan?
history	konsep penting dalam chat, streaming, dan progressive rendering yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana history diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **stream** -> **token** -> **typing state** -> **stop**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **stream** dapat memengaruhi **token**, lalu mengubah cara **typing state** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan chat, streaming, dan progressive rendering dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk stream serta token.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan chat, streaming, dan progressive rendering secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **stream** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan chat, streaming, dan progressive rendering.
- Menganggap stream sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan stream dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara token dan typing state dalam bab ini?
3. Sebutkan satu risiko ketika chat, streaming, dan progressive rendering diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan stream, token, typing state, stop. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 6 - Upload Gambar, Audio, dan Dokumen

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran upload gambar, audio, dan dokumen dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Upload Gambar, Audio, dan Dokumen tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
file input	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana file input diukur, diuji, atau dikendalikan?
preview	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana preview diukur, diuji, atau dikendalikan?
validation	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana validation diukur, diuji, atau dikendalikan?
progress	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana progress diukur, diuji, atau dikendalikan?
metadata	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana metadata diukur, diuji, atau dikendalikan?
consent	konsep penting dalam upload gambar, audio, dan dokumen yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana consent diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **file input** -> **preview** -> **validation** -> **progress**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **file input** dapat memengaruhi **preview**, lalu mengubah cara **validation** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan upload gambar, audio, dan dokumen dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk file input serta preview.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan upload gambar, audio, dan dokumen secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **file input** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan upload gambar, audio, dan dokumen.
- Menganggap file input sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan file input dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara preview dan validation dalam bab ini?
3. Sebutkan satu risiko ketika upload gambar, audio, dan dokumen diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan file input, preview, validation, progress. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 7 - Accessibility dan Internationalization

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran accessibility dan internationalization dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Accessibility dan Internationalization tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
keyboard	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana keyboard diukur, diuji, atau dikendalikan?
screen reader	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana screen reader diukur, diuji, atau dikendalikan?
contrast	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana contrast diukur, diuji, atau dikendalikan?
focus	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana focus diukur, diuji, atau dikendalikan?
language	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana language diukur, diuji, atau dikendalikan?
locale	konsep penting dalam accessibility dan internationalization yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana locale diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **keyboard** -> **screen reader** -> **contrast** -> **focus**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **keyboard** dapat memengaruhi **screen reader**, lalu mengubah cara **contrast** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan accessibility dan internationalization dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk keyboard serta screen reader.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan accessibility dan internationalization secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **keyboard** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan accessibility dan internationalization.
- Menganggap keyboard sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan keyboard dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara screen reader dan contrast dalam bab ini?
3. Sebutkan satu risiko ketika accessibility dan internationalization diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan keyboard, screen reader, contrast, focus. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 8 - Keamanan dan Privasi di Browser

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran keamanan dan privasi di browser dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Keamanan dan Privasi di Browser tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
XSS	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana XSS diukur, diuji, atau dikendalikan?
CSRF	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana CSRF diukur, diuji, atau dikendalikan?
token	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana token diukur, diuji, atau dikendalikan?
CSP	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana CSP diukur, diuji, atau dikendalikan?
PII	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana PII diukur, diuji, atau dikendalikan?
local storage	konsep penting dalam keamanan dan privasi di browser yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana local storage diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **XSS -> CSRF -> token -> CSP**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **XSS** dapat memengaruhi **CSRF**, lalu mengubah cara **token** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan keamanan dan privasi di browser dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk XSS serta CSRF.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan keamanan dan privasi di browser secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **XSS** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan keamanan dan privasi di browser.
- Menganggap XSS sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan XSS dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara CSRF dan token dalam bab ini?
3. Sebutkan satu risiko ketika keamanan dan privasi di browser diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan XSS, CSRF, token, CSP. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 9 - Testing dan Performa Front-end

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran testing dan performa front-end dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Testing dan Performa Front-end tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
unit	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana unit diukur, diuji, atau dikendalikan?
integration	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana integration diukur, diuji, atau dikendalikan?
end-to-end	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana end-to-end diukur, diuji, atau dikendalikan?
bundle	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana bundle diukur, diuji, atau dikendalikan?
lazy load	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana lazy load diukur, diuji, atau dikendalikan?
Core Web Vitals	konsep penting dalam testing dan performa front-end yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana Core Web Vitals diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **unit** -> **integration** -> **end-to-end** -> **bundle**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **unit** dapat memengaruhi **integration**, lalu mengubah cara **end-to-end** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan testing dan performa front-end dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk unit serta integration.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan testing dan performa front-end secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **unit** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan testing dan performa front-end.
- Menganggap unit sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan unit dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara integration dan end-to-end dalam bab ini?
3. Sebutkan satu risiko ketika testing dan performa front-end diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan unit, integration, end-to-end, bundle. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 10 - Design System dan Product Analytics

Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran design system dan product analytics dalam Front-end Development for AI Products;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

Gambaran Sederhana

Design System dan Product Analytics tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

Analogi: Front-end adalah ruang percakapan antara pengguna dan sistem. Desain yang baik bukan hanya terlihat rapi, tetapi membuat kemampuan, batasan, dan status AI dapat dipahami.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

Konsep Inti

Istilah	Penjelasan mudah	Cara memeriksa pemahaman
token	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana token diukur, diuji, atau dikendalikan?
component library	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana component library diukur, diuji, atau dikendalikan?
consistency	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana consistency diukur, diuji, atau dikendalikan?
event	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana event diukur, diuji, atau dikendalikan?
funnel	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana funnel diukur, diuji, atau dikendalikan?
experimentation	konsep penting dalam design system dan product analytics yang perlu diberi definisi operasional sebelum dipakai	Pertanyaan yang perlu dijawab: bagaimana experimentation diukur, diuji, atau dikendalikan?

Hubungan antarkonsep

Bayangkan alur **token -> component library -> consistency -> event**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **token** dapat memengaruhi **component library**, lalu mengubah cara **consistency** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

Langkah Kerja

1. Tentukan tujuan design system dan product analytics dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk token serta component library.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

Contoh Kasus

Dalam mini project **Dashboard Evaluasi Diri Berbasis Umpan Balik**, tim perlu menerapkan design system dan product analytics secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **token** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

Situasi	Pilihan tindakan	Trade-off
Data atau input belum lengkap	Tolak, minta perbaikan, atau gunakan fallback	Menjaga kualitas tetapi dapat menambah friksi
Hasil belum pasti	Tampilkan ketidakpastian dan minta review	Lebih aman tetapi membutuhkan waktu manusia
Beban meningkat	Scale, antrekan, atau pembatasan	Menjaga layanan tetapi menambah biaya atau waktu tunggu
Perubahan berisiko	Uji terbatas dan siapkan rollback	Rilis lebih lambat tetapi dampak kegagalan lebih kecil

Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan design system dan product analytics.
- Menganggap token sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

Checkpoint

1. Jelaskan token dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara component library dan consistency dalam bab ini?
3. Sebutkan satu risiko ketika design system dan product analytics diterapkan tanpa pengujian.

Latihan

1. Buat diagram sederhana yang menghubungkan token, component library, consistency, event. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

Bab 11 - Mini Project: Dashboard Evaluasi Diri Berbasis Umpan Balik

Tujuan Proyek

Membangun antarmuka formulir dan dashboard yang mengumpulkan penilaian orang lain, menampilkan pola umpan balik, serta menjaga anonimitas dan aksesibilitas.

Proyek harus cukup kecil untuk selesai, tetapi cukup lengkap untuk memperlihatkan alur berpikir. Nilai utama bukan pada tampilan atau kompleksitas, melainkan kejelasan tujuan, kualitas keputusan, pengujian, dan dokumentasi.

Deliverable

- problem statement dan intended use;
- diagram arsitektur atau alur kerja;
- data dictionary atau inventaris sumber;
- baseline yang dapat dijalankan;
- hasil pengujian normal, error, dan kasus batas;
- daftar risiko serta kontrol;
- ringkasan hasil untuk audiens nonteknis;
- README dan rencana pengembangan.

Tahapan Pengerjaan

1. Tulis masalah, pengguna, keputusan, dan batas penggunaan.
2. Buat inventaris data, sumber daya, pemilik, izin, serta risiko.
3. Rancang arsitektur atau alur kerja versi minimum.
4. Bangun baseline yang dapat dijalankan ulang.
5. Tambahkan validasi, logging, dan penanganan error.
6. Susun golden set atau skenario uji, termasuk kasus batas.
7. Ukur outcome, guardrail, performa, dan biaya.
8. Dokumentasikan hasil, keterbatasan, runbook, serta rencana iterasi.

Contoh Kode Awal

```
const controller = new AbortController();

async function requestInsight(payload) {
  const response = await fetch("/api/insights", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(payload),
    signal: controller.signal
  });
  if (!response.ok) throw new Error(`Request gagal: ${response.status}`);
  return response.json();
}

document.querySelector("#stop").onclick = () => controller.abort();
```

Kode tersebut hanya titik awal. Tambahkan pemeriksaan input, konfigurasi, test, logging, serta penanganan error sesuai konteks proyek.

Rubrik Penilaian

Aspek	Bobot	Indikator
Problem framing	15%	Pengguna, keputusan, outcome, dan batas jelas
Data atau input	15%	Sumber, izin, kualitas, dan representasi dijelaskan
Rancangan	15%	Komponen, interface, serta trade-off

Aspek	Bobot	Indikator
		masuk akal
Implementasi	20%	Baseline berjalan, modular, dan dapat diulang
Evaluasi	15%	Metrik, kasus batas, serta subgroup diperiksa
Responsible practice	10%	Privasi, keamanan, fairness, dan human review
Komunikasi	10%	Dokumentasi dan demo mudah dipahami

Pertanyaan Review Proyek

1. Keputusan apa yang benar-benar dibantu proyek ini?
2. Apa kegagalan yang paling mungkin dan paling berbahaya?
3. Bukti apa yang menunjukkan solusi lebih baik daripada baseline?
4. Siapa yang berwenang menyetujui, menghentikan, atau mengubah sistem?
5. Informasi apa yang harus terlihat oleh pengguna agar tidak salah memahami hasil?

Bab 12 - Kuis Akhir

Pilih jawaban yang paling tepat.

1. Ketika memulai **Peran Front-end dan Karakteristik AI UX**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji interface
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
2. Ketika memulai **Fondasi HTML, CSS, dan JavaScript**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji semantic HTML
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
3. Ketika memulai **Component, State, dan Data Flow**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji component
 - D. Mengukur keberhasilan hanya dari jumlah fitur
4. Ketika memulai **Integrasi API dan Async Interaction**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji fetch
5. Ketika memulai **Chat, Streaming, dan Progressive Rendering**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji stream
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
6. Ketika memulai **Upload Gambar, Audio, dan Dokumen**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji file input
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
7. Ketika memulai **Accessibility dan Internationalization**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mendefinisikan tujuan, konteks, dan cara menguji keyboard
 - D. Mengukur keberhasilan hanya dari jumlah fitur
8. Ketika memulai **Keamanan dan Privasi di Browser**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Menghapus seluruh review agar proses lebih cepat
 - C. Mengukur keberhasilan hanya dari jumlah fitur
 - D. Mendefinisikan tujuan, konteks, dan cara menguji XSS
9. Ketika memulai **Testing dan Performa Front-end**, tindakan yang paling tepat adalah ...
 - A. Mendefinisikan tujuan, konteks, dan cara menguji unit
 - B. Memilih alat paling baru tanpa memeriksa kebutuhan
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur
10. Ketika memulai **Design System dan Product Analytics**, tindakan yang paling tepat adalah ...
 - A. Memilih alat paling baru tanpa memeriksa kebutuhan
 - B. Mendefinisikan tujuan, konteks, dan cara menguji token
 - C. Menghapus seluruh review agar proses lebih cepat
 - D. Mengukur keberhasilan hanya dari jumlah fitur

11. Input penting berubah tanpa pemberitahuan. Kontrol terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Kontrak, validasi, versioning, dan alert
 - D. menghapus logging untuk menghemat ruang
12. Hasil sistem terlihat baik pada data latihan tetapi buruk pada konteks baru. Langkah terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Memeriksa split, leakage, representasi data, dan generalisasi
13. Tim tidak tahu siapa yang harus merespons alarm. Perbaikan utama adalah ...
- A. Menetapkan ownership, severity, runbook, dan escalation
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
14. Perubahan besar akan diluncurkan. Cara menurunkan blast radius adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Rilis bertahap, observasi, dan rollback
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
15. Pengguna tidak memahami keterbatasan hasil. Desain yang tepat adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Menjelaskan sumber, ketidakpastian, batas penggunaan, dan koreksi
 - D. menghapus logging untuk menghemat ruang
16. Data sensitif digunakan untuk latihan. Prinsip pertama adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Minimization, izin, kontrol akses, dan penggunaan data sintetis bila memungkinkan
17. Satu metrik meningkat tetapi dampak keseluruhan memburuk. Tim perlu ...
- A. Menggunakan outcome metric dan guardrail secara bersamaan
 - B. menambah kompleksitas tanpa diagnosis
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
18. Sistem gagal pada sebagian kelompok. Respons yang benar adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. Menganalisis subgroup, dampak, akar masalah, dan mitigasi
 - C. menyembunyikan error dari pengguna
 - D. menghapus logging untuk menghemat ruang
19. Biaya naik tanpa kenaikan nilai. Langkah awal adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. Mengukur penggunaan per komponen dan menghubungkannya dengan outcome
 - D. menghapus logging untuk menghemat ruang
20. Temuan belum cukup kuat untuk keputusan otomatis. Pilihan aman adalah ...
- A. menambah kompleksitas tanpa diagnosis
 - B. menyembunyikan error dari pengguna
 - C. menghapus logging untuk menghemat ruang
 - D. Membatasi penggunaan sebagai dukungan keputusan dan menambahkan human review

Kunci dan Pembahasan

No.	Jawaban	Pembahasan
1	A	Pendekatan dimulai dari tujuan dan

No.	Jawaban	Pembahasan
		definisi operasional interface, kemudian alat dipilih sesuai kebutuhan.
2	B	Pendekatan dimulai dari tujuan dan definisi operasional semantic HTML, kemudian alat dipilih sesuai kebutuhan.
3	C	Pendekatan dimulai dari tujuan dan definisi operasional component, kemudian alat dipilih sesuai kebutuhan.
4	D	Pendekatan dimulai dari tujuan dan definisi operasional fetch, kemudian alat dipilih sesuai kebutuhan.
5	A	Pendekatan dimulai dari tujuan dan definisi operasional stream, kemudian alat dipilih sesuai kebutuhan.
6	B	Pendekatan dimulai dari tujuan dan definisi operasional file input, kemudian alat dipilih sesuai kebutuhan.
7	C	Pendekatan dimulai dari tujuan dan definisi operasional keyboard, kemudian alat dipilih sesuai kebutuhan.
8	D	Pendekatan dimulai dari tujuan dan definisi operasional XSS, kemudian alat dipilih sesuai kebutuhan.
9	A	Pendekatan dimulai dari tujuan dan definisi operasional unit, kemudian alat dipilih sesuai kebutuhan.
10	B	Pendekatan dimulai dari tujuan dan definisi operasional token, kemudian alat dipilih sesuai kebutuhan.
11	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
12	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
13	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
14	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
15	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
16	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

No.	Jawaban	Pembahasan
17	A	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
18	B	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
19	C	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.
20	D	Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.

Interpretasi Skor

- **17-20 benar:** siap mengerjakan proyek dengan pengawasan ringan.
- **13-16 benar:** pemahaman baik, ulangi bagian yang masih lemah.
- **9-12 benar:** pelajari kembali konsep inti dan latihan.
- **0-8 benar:** ulangi modul secara bertahap dengan mentor atau teman belajar.

Bab 13 - Diskusi dan Refleksi

Pertanyaan Diskusi

1. Bagian mana dari Front-end Development for AI Products yang paling mudah diremehkan tetapi berpotensi menimbulkan kegagalan besar?
2. Kapan solusi sederhana lebih baik daripada solusi yang lebih otomatis atau kompleks?
3. Metrik apa yang dapat mendorong perilaku salah bila digunakan sendirian?
4. Bagaimana pengguna dapat mengoreksi sistem dan melihat tindak lanjutnya?
5. Siapa yang menerima manfaat dan siapa yang mungkin menerima risiko?

Jurnal Refleksi

Tuliskan satu halaman yang menjawab:

- konsep yang paling mengubah cara berpikir;
- asumsi yang sebelumnya dianggap benar;
- kesalahan yang terjadi ketika latihan;
- bukti yang masih kurang;
- satu tindakan belajar berikutnya.

Bab 14 - Glosarium dan Checklist

Glosarium

Istilah	Makna ringkas
interface	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
feedback	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
uncertainty	tingkat ketidakpastian yang perlu ditampilkan dan dikelola.
control	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
trust	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
failure	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
semantic HTML	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
layout	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
responsive	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
DOM	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
event	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
module	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
component	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
props	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
state	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
derived state	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.

Istilah	Makna ringkas
lifecycle	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
architecture	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
fetch	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
promise	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
loading	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
timeout	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
retry	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
cancellation	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
stream	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
token	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
typing state	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
stop	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
regenerate	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
history	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
file input	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
preview	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
validation	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.

Istilah	Makna ringkas
progress	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
metadata	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
consent	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
keyboard	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
screen reader	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
contrast	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
focus	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
language	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
locale	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
XSS	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
CSRF	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.
CSP	konsep penting dalam front-end development for ai products yang perlu diberi definisi operasional sebelum dipakai.

Checklist Sebelum Implementasi

- ☐ Masalah, pengguna, dan keputusan telah dinyatakan.
- ☐ Intended use dan penggunaan yang dilarang telah ditulis.
- ☐ Sumber data atau input memiliki izin dan provenance.
- ☐ Baseline dan kriteria penerimaan tersedia.
- ☐ Kondisi error, data kosong, dan kasus ekstrem diuji.
- ☐ Privasi, keamanan, bias, dan aksesibilitas ditinjau.
- ☐ Log, metric, owner, alert, dan runbook ditetapkan.
- ☐ Perubahan memiliki versioning, approval, dan rollback.
- ☐ Hasil dikomunikasikan bersama keterbatasannya.
- ☐ Jadwal review dan penghentian sistem ditentukan.

Bab 15 - Ringkasan dan Referensi

Ringkasan Cepat

1. Mulai dari keputusan dan pengguna, bukan alat.
2. Definisikan istilah, data, outcome, dan guardrail secara operasional.
3. Bangun baseline kecil yang dapat diamati dan diuji ulang.
4. Perlakukan keamanan, privasi, aksesibilitas, serta human review sebagai bagian desain.
5. Uji kondisi normal, kegagalan, subgroup, perubahan konteks, dan biaya.
6. Dokumentasikan provenance, asumsi, versi, owner, serta keterbatasan.
7. Gunakan monitoring dan feedback untuk memutuskan iterasi, rollback, atau penghentian.

Referensi Utama

- MDN Web Docs.
- Web Content Accessibility Guidelines (WCAG).
- WAI-ARIA Authoring Practices.
- Web Security Academy - PortSwigger.
- Human-Centered AI - Ben Shneiderman.

Penutup

Kemampuan dalam Front-end Development for AI Products tidak hanya ditunjukkan oleh banyaknya alat yang dikuasai. Kemampuan terlihat dari cara peserta merumuskan masalah, memilih pendekatan yang proporsional, menguji klaim, melindungi pengguna, dan menjaga sistem tetap dapat dipertanggungjawabkan.

HerAI Fellowship - Participant Module Versi 1.0.0 - 23 Juli 2026